Homework 5 Conceptual

CSCI 1470/2470: Deep Learning

Due Monday, April 8th, 2024 at 6:00pm EST

1 Conceptual Questions

1. Question 1: Attention mechanisms can be used for both RNNs and Transformers.

   Is the statement true or false? Please explain/illustrate using lecture material. (4-5 sentences) Is there an advantage of using one framework over the other? (4-5 setences)

   [ANSWER] The statement is true. Attention mechanisms can be used in both Recurrent Neural Networks (RNNs) and Transformers. In RNNs, attention mechanisms can be added to enhance the model's ability to remember long sequences by selectively focusing on different parts of the input sequence for each output. This is particularly useful in tasks like machine translation, where the model needs to remember and relate different parts of the input sentence while generating the translation.Transformers, on the other hand, are built around the concept of self-attention and do not rely on recurrence to process sequences. The self-attention mechanism in Transformers allows each position in the sequence to attend to all positions in the previous layer of the model simultaneously, making them highly efficient for tasks involving long sequences and parallelizable computations. This design enables Transformers to achieve remarkable performance in a wide range of tasks, such as natural language understanding, generation, and even beyond text-based applications. While the use and implementation of attention mechanisms differ between RNNs and Transformers, both architectures can utilize attention to improve their performance on sequence-based tasks.

2. Question 2: What is the purpose of the positional encoding in the Transformer architecture? What is the size of a positional encoding vector and how is it calculated (on a high-level, refer lecture slides)? (2-4 sentences)

[**ANSWER**] The purpose of positional encoding in the Transformer architecture is to provide a means of incorporating the order of the sequence into the model, as Transformers do not inherently process sequential data in order like RNNs do. Positional encodings are added to the input embeddings to give the model information about the position of each word (or token) in the sequence. The size of a positional encoding vector is the same as the size of the input embedding vector, ensuring that the two can be summed directly. At the high level, positional encodings are calculated using a specific function of the position and the dimension of the embeddings, involving sinusoidal patterns for different frequencies, allowing the model to distinguish the position of each token within the sequence.

3. Question 3 : Consider the parameters for two different attention heads. Is it necessary that they be initialized randomly, or could we just start them all with the same vector of values? (2-4 sentences)

[ANSWER] In the Transformer architecture, it's crucial that the parameters for different attention heads are initialized randomly rather than starting them all with the same vector of values. This random initialization ensures that each attention head can learn to focus on different parts of the input sequence, thereby capturing various aspects of the input data. If all attention heads were initialized identically, they would initially produce identical outputs and gradients during training, which could prevent them from diversifying and learning different representations. Random initialization helps in breaking this symmetry and enables the model to explore a richer space of functions by allowing each head to evolve independently during the learning process.

4. Question 4 : Suppose we are in the encoder block learning the embedding for the word "Thinking" (in the input sentence "Thinking Machine") after applying self-attention. See figure below. What will be the final output given the values for queries, keys, and values for both the words in the sentence? Show all the calculation steps. At the softmax stage and after, use three decimal points. Remember: For this question, the self attention final output (z1 in the figure) is being calculated only for the word "Thinking" with respect to the sentence "Thinking Machine". (Optinal) For bonus points Calculate z2 , final output for "Machine" with respect to the sentence.

[**ANSWER**]
Here are the calculation steps:
1. The scores are computed by taking the dot product of the query vectors with the key vectors. $q1 \cdot k1$ and $q1 \cdot k2$ are 6 and 14, respectively.
2. After dividing by $\sqrt{d_k}$ (where d_k is the dimension of the key vectors, which is 3 in this case) and rounding to the nearest whole number, the normalized scores are 3 and 7.
3. The softmax scores:

$$\text{softmax}(q1 \cdot k1) = \frac{e^3}{e^3 + e^7}$$

$$\text{softmax}(q1 \cdot k2) = \frac{e^7}{e^3 + e^7}$$

Then the softmax probabilities are:

$$\text{softmax}(q1 \cdot k1) \approx \frac{20.085}{1116.718} \approx 0.018$$

$$\text{softmax}(q1 \cdot k2) \approx \frac{1096.633}{1116.718} \approx 0.982$$

4.
The value vectors $v1$ and $v2$ are:

$$v1 = [2, 41, 3]$$

$$v2 = [9, 0, 13]$$

So that our z1 calculation is:

$$z1 = 0.018 \times [2, 41, 3] + 0.982 \times [9, 0, 13]$$

$$z1 = [0.036, 0.738, 0.054] + [8.838, 0, 12.766]$$

Rounding these to three decimal points, the elements of vector $z1$ are approximately:

$$z1 \approx [8.874, 0.738, 12.820]$$

This vector $z1$ represents the output of the self-attention mechanism for the word "Thinking" in the context of the sentence.

5. Question 5 : Now suppose there is a decoder block to translate the example sentence "Thinking Machine" above into "Machine à penser" how will it be different from the encoder block? Explain using 4-5 sentences.

[ANSWER] A decoder block in the context of translating "Thinking Machine" into "Machine à penser" operates differently from an encoder block in several key aspects, despite some structural similarities:
1. Input and Attention: While both encoder and decoder blocks use self-attention mechanisms, the decoder block also incorporates an additional mechanism called "encoder-decoder attention." In this mechanism, the decoder uses its output up to the current position to query the entire output of the encoder, allowing it to focus on relevant parts of the input sentence when producing each word in the translation.
2. Masked Self-Attention: The self-attention mechanism in the decoder is "masked" to prevent positions from attending to subsequent positions. This ensures that the prediction for a certain position can only depend on known outputs at positions before it, maintaining the auto-regressive property required for generating text one token at a time.
3. Sequential Generation: The decoder generates the translation sequentially, taking into account the previously generated tokens (in this case, in French) and the entire input sequence's representation from the encoder to decide on the next token. This is in contrast to the encoder, which processes the entire input sequence simultaneously.
4. Language Specificity: In the decoder, the final layers (before generating predictions for the next token) are specifically tailored to the target language, including language-specific embeddings and potentially a language model, to more accurately predict the sequence of the target language ("Machine à penser" in this case).
Overall, while the encoder captures the context of the entire input sequence, the decoder focuses on generating the correct target sequence one token at a time, guided by the encoder's output and what it has produced so far.

2 2470-only Questions

1. Question 1: What requires more parameters: single or multi-headed attention? Explain. Does this mean one necessarily trains faster than another?

[ANSWER]

Parameter Requirements

Single-Headed Attention computes the attention scores between pairs of elements in the input sequence in one "head." The computation involves three sets of weights (queries, keys, and values) and typically a softmax operation to obtain attention scores that sum to 1. The number of parameters in a single-headed attention mechanism depends on the dimensionality of these weights, which in turn depends on the dimensionality of the input and output vectors.

Multi-Headed Attention, on the other hand, divides the input vectors into multiple "heads," allowing the model to simultaneously attend to information from different representation subspaces at different positions. Each head has its set of query, key, and value weight matrices, so the total number of parameters in a multi-headed attention mechanism is roughly the number of heads times the number of parameters in a single-headed attention mechanism. This is because each head will have its own set of weights, although each set might be smaller compared to the single-head scenario, depending on how the model is designed.

Thus, multi-headed attention requires more parameters than single-headed attention because it replicates the attention mechanism across multiple heads, each with its own set of parameters.

Training Speed Consideration

However, having more parameters does not necessarily mean one trains faster than the other. The training speed can be influenced by several factors:

Parallelization: Multi-headed attention mechanisms can leverage parallel processing capabilities of modern hardware (like GPUs) more effectively than single-headed attention because each head can be processed in parallel.

Efficiency in Learning Representations: Multi-headed attention might learn richer representations more efficiently, which can sometimes lead to faster convergence despite the higher number of parameters.

Model Complexity and Data Requirements: A more complex model with more parameters might require more data and more training iterations to converge, potentially slowing down training compared to a simpler model.

2. Question 2: Transformers can also be used for language modeling. In fact, they are the current state-of-the-art method used. (see https://openai.com/blog/better-language-models/). How are transformer-based language models similiar to convolution? What makes them more suited for language modeling?

[**ANSWER**]
Similarities to Convolution
1. Feature Extraction: Both CNNs and transformers are capable of extracting features from their input data. CNNs do this through convolutional filters that detect patterns such as edges in images, whereas transformers use self-attention mechanisms to identify relationships and patterns in the data, like the syntactical structure in text.
2. Hierarchical Processing: Just as CNNs can build a hierarchical representation of visual data, with lower layers detecting simple patterns and higher layers understanding complex structures, transformers process data in layers where each layer can learn to focus on different aspects of the input data through the self-attention mechanism.
3. Learning Contextual Representations: In the context of language, CNNs can be designed to understand contextual information within a fixed window size (e.g., n-gram modeling in text), capturing local dependencies. Similarly, transformers learn contextual representations, but with a crucial advantage: their attention mechanisms can capture dependencies regardless of their distance in the input sequence, providing a broader and more flexible context understanding.
Advantages of Transformers for Language Modeling
Transformers are more suited for language modeling for several reasons:
1. Long-range Dependencies: One of the key strengths of transformers over CNNs is their ability to handle long-range dependencies in text. The self-attention mechanism allows each position in the input sequence to attend to every other position, directly capturing relationships between elements far apart in the sequence. This is particularly beneficial for understanding the structure and nuances of natural language.
2. Parallel Processing: Unlike CNNs and RNNs, transformers process all elements of the input sequence in parallel rather than sequentially. This parallelism significantly reduces training times and allows the model to scale with the amount of available computational resources.
3. Flexibility and Scalability: Transformers have shown remarkable flexibility and scalability, performing well across different languages, domains, and tasks (e.g., translation, summarization, question-answering). Their architecture can be efficiently scaled up (with more layers or larger hidden sizes) to improve performance, a property that has led to the development of massive models like GPT series that achieve state-of-the-art results in language modeling.
4. Dynamic Attention: Unlike the static nature of convolutional filters, the attention weights in transformers are dynamically computed for each input. This means that the model can adaptively focus on different parts of the input sequence depending on the context, leading to a more nuanced understanding of language.

3. Question 3: Read about BERT, a former state-of-the-art transformer-based language model here: https://arxiv.org/pdf/1810.04805.pdf, and answer the following questions.

   a) What did the researchers claim was novel about BERT? Why was this better than previous forms of language modeling techniques?

   b) What was the masked language model objective? Describe this in 1-2 sentences.

   c) Pretraining and finetuning both are forms of training a model. What's the difference between pretraining and finetuning, and how does BERT use both techniques?

[**ANSWER**] a) The researchers claimed that BERT is novel because it introduced a bidirectional training objective for language models, allowing it to capture context from both left and right directions. This is in contrast to previous language modeling techniques, which were either unidirectional or used a combination of left-to-right and right-to-left models. BERT's bidirectional training objective enables it to better understand the context and meaning of words in a sentence. Additionally, BERT introduced the use of masked language modeling, where a certain percentage of words in a sentence are masked and the model is trained to predict the masked words. This helps BERT learn deeper contextual representations. Overall, BERT's novel approach to training language models resulted in significant improvements in various natural language processing tasks.
b) The masked language model objective in BERT is a pre-training task where some of the tokens in the input are randomly masked, and the model is trained to predict the original vocabulary id of the masked word based on its context. This allows BERT to learn bidirectional representations by fusing the left and right context.
c) Pretraining and finetuning are two different stages in training a model. Here is the difference between the two and how BERT utilizes both techniques:

(a) Pretraining: Pretraining is the initial stage where the model is trained on a large amount of unlabeled data. During pretraining, BERT uses two unsupervised tasks: masked language modeling (MLM) and next sentence prediction (NSP). MLM randomly masks some tokens from the input and the model predicts the original vocabulary id of the masked word based on its context. NSP is a binary classification task where the model predicts whether two sentences are consecutive or not. Pretraining allows BERT to learn general language representations by capturing bidirectional context information.

(b) Finetuning: Finetuning is the second stage where the pretrained model is further trained on a specific downstream task using labeled data. BERT takes the pretrained parameters and fine-tunes them by updating all the parameters during finetuning. The architecture of BERT remains the same during finetuning, but the task-specific inputs and outputs are plugged into the model. BERT can be finetuned for various tasks such as question answering, natural language inference, and sentiment analysis. Finetuning allows BERT to adapt its pretrained representations to the specific task at hand.

In summary, BERT uses pretraining to learn general language representations by training on unlabeled data, and then uses finetuning to adapt those representations to specific downstream tasks by training on labeled data. This two-step process allows BERT to achieve state-of-the-art performance on a wide range of natural language processing tasks.